Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

Department of Information Technology (B.Sc.I.T Semester V)

Dot .Net Core and Progressive Web Application Practical

Practical – III

Roll No.: T010	Name: Sagar Kandari
Class: TYIT	Batch:1
Date of Assignment: 11/07/2026	Date/Time of Submission: 11/07/2026

a) Create methods add(). multiply(), substract() ,divide() with suitable parameters and call these methods using concept of C# delegate.

ALGORITHM:-

Step 1 - Start.

Step 2 - Declare a delegate with two integer parameters.

Step 3 - Create methods:

Add(int a, int b)

Subtract(int a, int b)

Multiply(int a, int b)

Divide(int a, int b)

Step 4 - In the Main() method:

Create a delegate object.

Assign the Add() method to the delegate and call it.

Assign the Subtract() method to the delegate and call it.

Assign the Multiply() method to the delegate and call it.

Assign the Divide() method to the delegate and call it.

Step 5 - Display the results.

Step 6 - Stop.

Code:

```
using System;

delegate void Operation(int a, int b);

class Calculator
{
 public void Add(int a, int b)
 {
 Console.WriteLine("Addition = " + (a + b));
 }

 public void Subtract(int a, int b)
 {
 Console.WriteLine("Subtraction = " + (a - b));
 }

 public void Multiply(int a, int b)
 {
 Console.WriteLine("Multiplication = " + (a * b));
 }

 public void Divide(int a, int b)
 {
 Console.WriteLine("Division = " + (a / b));
 }
}


class Program
{
 static void Main(string[] args)
 {
 Calculator c = new Calculator();

 Operation opAdd = c.Add;
 opAdd(20, 10);

 Operation opSub = c.Subtract;
 opSub(30, 10);

 Operation opMul = c.Multiply;
 opMul(10, 5);

 Operation opDiv = c.Divide;
 opDiv(10, 5);
 Console.WriteLine("sagar");
 }
}
```

Output:

```
Addition = 30
Subtraction = 20
Multiplication = 50
Division = 2
sagar
```

b) Write a program using multicast delegate.

ALGORITHM:-

Step 1 - Start.

Step 2- Declare a delegate with no parameters.

Step 3 - Create three methods (e.g., Welcome(), Learn(), ThankYou()).

Step 4 - In the Main() method:

Create a delegate object.

Assign the first method to the delegate.

Add the remaining methods using the += operator.

Invoke the delegate object.

Step 5 - All methods execute one after another.

Step 6 - Stop.

Code:

```
using System;

delegate void Message();

class Demo
{
 public void Welcome()
 {
 Console.WriteLine("Welcome Students");  }
 public void Learn()
 {
 Console.WriteLine("Learning C# delegate");
 }
}
```

```
class Program
{
 static void Main(string[] args)
 {
 Demo d = new Demo();

 Message msg = d.Welcome;
 msg += d.Learn;

 msg();

 Console.WriteLine("sagar");
 }
}
```

Output:

```
Welcome Students
Learning C# delegate
sagar
```

c) Create a class BankAccount with AccountNumber and Balance. Implement property for Balance with validation (Balance cannot be negative).

ALGORITHM:-

Step 1 - Start.

Step 2 - Create a class BankAccount.

Step 3 - Declare private data members:

AccountNumber

Balance

Step 4 - Create a property for Balance.

Step 5 - In the set accessor:

Check whether the assigned value is greater than or equal to 0.

If yes, store the value in Balance.

Otherwise, display an error message: "Balance cannot be negative."

Step 6 - Create an object of the BankAccount class in the Main() method.

Step 7 - Assign the account number.

Step 8 - Assign a valid balance and display the account details.

Step 9 - If a negative balance is entered, display the validation message.

Step 10 - Stop.

Code:

```
using System;

class BankAccount
{
 public int AccountNumber { get; set; }

 private decimal balance;

 public decimal Balance
 {
 get { return balance; }
 set
 {
 if (value < 0)
 {
 Console.WriteLine("Balance cannot be negative!");
 }
 else
 {
 balance = value;
 }
 }
 }
}

class Program
{
 static void Main(string[] args)
 {
 BankAccount account = new BankAccount();
 account.AccountNumber = 101;

 account.Balance = 5000;
 Console.WriteLine("Account Number: " + account.AccountNumber);
Console.WriteLine("Balance: " + account.Balance);

 account.Balance = -200;
 Console.WriteLine("Balance after invalid update: " + account.Balance);
Console.WriteLine("sagar");
 }
}
```

Output:

```
Account Number: 101
Balance: 5000
Balance cannot be negative!
Balance after invalid update: 5000
sagar
```